

IAM-07: Audit Logging and Compliance

Series: IAM — IAM Administration | **Notebook:** 7 of 12 | **Created:** January 2026 |
Last Updated: 08/12/2026

Meeting Regulatory and Security Requirements

Audit logging is essential for compliance (SOC2, SOX, HIPAA, PCI-DSS) and security operations. This notebook covers querying audit logs, building compliance reports, and conducting access reviews.

Table of Contents

- 1. [Audit Log Fundamentals](#)
- 2. [Authentication Event Monitoring](#)
- 3. [Authorization and Access Events](#)
- 4. [Configuration Change Tracking](#)
- 5. [Compliance Reporting Patterns](#)
- 6. [Access Review Workflows](#)
- 7. [Building Compliance Dashboards](#)

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Gen3 IAM and audit logging enabled

Requirement	Details
Permissions	<code>logs.read</code> with access to audit log bucket
Data Retention	Audit logs retained per compliance requirements

1. Audit Log Fundamentals

Dynatrace audit logs capture all security-relevant events in your environment.

What Gets Logged

Category	Events
Authentication	Logins, logouts, MFA events, SSO
Authorization	Access attempts, permission checks
User Management	User creation, modification, deletion
Group Management	Group changes, membership updates
Policy Changes	Policy creation, modification, deletion
Token Operations	Token creation, revocation, usage
Configuration	Settings changes, API calls

Audit Log Fields

Field	Description	Example
<code>timestamp</code>	Event time	<code>2026-01-26T14:30:00Z</code>
<code>log.source</code>	Log category	<code>audit</code>
<code>content</code>	Event details	JSON with action details
<code>dt.security.user</code>	Acting user	<code>user@company.com</code>
<code>dt.source_entity</code>	Source entity	Client IP, service

Retention Requirements

Compliance	Minimum Retention
SOC2	1 year
SOX	7 years
HIPAA	6 years
PCI-DSS	1 year online, 1+ archive
GDPR	As long as necessary

2. Authentication Event Monitoring

Track who is accessing your Dynatrace environment and how.

```
// All authentication events in the last 24 hours
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "AUDIT_EVENT"
| filter isNotNull(authentication.type)
| summarize events = count(), by:{authentication.type, event.outcome}
| sort events desc
```

```
// Failed login attempts - critical for security monitoring
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter not startsWith(event.outcome, "2") and event.outcome != "success"
| summarize failures = count(), by:{user.id, event.outcome}
| sort failures desc
| limit 25
```

```
// Login activity by hour to identify patterns
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter isNotNull(authentication.type)
| makeTimeseries logins = count(), interval:1h
```

Authentication Patterns to Monitor

Pattern	Indicates	Query Focus
Multiple failed logins	Brute force attempt	Failed + same user
Off-hours logins	Unusual access	Timestamp analysis
New location logins	Account compromise	IP/location change
Rapid successive logins	Automation or sharing	Time between logins
Login after offboarding	Process failure	User status check

```
// Off-hours activity detection (outside 08:00-18:00 UTC)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| fieldsAdd hour = formatTimestamp(timestamp, format: "HH")
| filter toLong(hour) < 8 or toLong(hour) >= 18
| summarize off_hours_events = count(), by:{user.id}
| sort off_hours_events desc
| limit 25
```

3. Authorization and Access Events

Track permission checks and access attempts to sensitive resources.

```
// Access denied events - indicates permission issues or attacks
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter event.outcome == "403"
| summarize denials = count(), by:{user.id, resource}
| sort denials desc
| limit 25
```

```
// Permission changes and grants
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "iam")
| summarize changes = count(), by:{user.id, event.type}
| sort changes desc
```

Access Event Categories

Event Type	Security Relevance	Monitoring Priority
Access denied	Permission misconfiguration or probe	High
Privilege escalation	Critical security event	Critical
Cross-environment access	Boundary violation	High
Admin action	Privileged operation	Medium
Bulk data access	Exfiltration risk	High

4. Configuration Change Tracking

Monitor all changes to IAM configuration.

```
// User creation and deletion events
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"CREATE", "DELETE"})
| fields timestamp, user.id, event.type, resource, event.outcome
| sort timestamp desc
| limit 50
```

```

// Group membership changes
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "iam")
| fields timestamp, user.id, event.type, resource
| sort timestamp desc
| limit 50

```

```

// Policy changes - critical for compliance
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "iam")
| summarize policy_changes = count(), by:{user.id}
| sort policy_changes desc

```

```
// Token creation and revocation
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter contains(resource, "token") and in(event.type, {"CREATE", "DELETE",
"POST"})
| fields timestamp, user.id, event.type, resource, event.outcome
| sort timestamp desc
| limit 50
```

Change Tracking Summary

Change Type	Audit Requirement	Review Frequency
User lifecycle	Log all	Weekly
Group changes	Log all	Weekly
Policy changes	Log + approve	On change
Admin actions	Log + review	Daily
Token operations	Log all	Monthly

5. Compliance Reporting Patterns

Standard queries for compliance frameworks.

SOC2 - Access Control Evidence

SOC2 requires evidence of access controls and monitoring.

```
// SOC2: User access review - all users who accessed system in period
fetch logs, from: now() - 90d
| filter matchesPhrase(log.source, "audit")
| filter matchesPhrase(content, "login")
| summarize last_login = max(timestamp), login_count = count(), by: { content
}
| sort last_login desc
| limit 500
```

```
// SOC2: Administrative action log
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-90d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"})
| fields timestamp, user.id, user.organization, event.type, resource,
event.outcome
| sort timestamp desc
| limit 100
```

SOX - Segregation of Duties

SOX requires segregation of duties and change authorization.

```

// SOX: Privileged user activity (write operations)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"})
| summarize privileged_actions = count(), by:{user.id, event.type}
| sort privileged_actions desc
| limit 25

```

```
// SOX: Configuration changes requiring approval
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"}) and contains(resource, "settings")
| fields timestamp, user.id, event.type, resource, event.outcome
| sort timestamp desc
| limit 50
```

HIPAA - PHI Access Logging

HIPAA requires logging of access to protected health information.

```
// HIPAA: All data access events
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter event.type == "GET"
| summarize reads = count(), by:{user.id, resource}
| sort reads desc
| limit 50
```

```
// HIPAA: Access anomalies - bulk access detection
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| summarize requests = count(), by:{user.id}
| filter requests > 1000
| sort requests desc
| limit 25
```

PCI-DSS - Cardholder Data Access

PCI-DSS requires tracking access to cardholder data.

```
// PCI-DSS: Failed access attempts (potential attack indicator)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter not startsWith(event.outcome, "2") and event.outcome != "success"
| summarize failures = count(), by:{user.id, origin.address, event.outcome}
| sort failures desc
| limit 50
```

Compliance Query Reference

Framework	Key Reports	Frequency
SOC2	User access, admin actions, logical access	Quarterly
SOX	Privileged users, change authorization	Quarterly
HIPAA	PHI access, anomaly detection	Monthly
PCI-DSS	Failed access, CDE access	Daily/Weekly
GDPR	Data access, consent tracking	On request

6. Access Review Workflows

Periodic access reviews are required by most compliance frameworks.

Access Review Process

Step	Action	Owner
1	Generate user access report	IAM Admin
2	Distribute to managers	IAM Admin
3	Review and certify access	Managers
4	Remove inappropriate access	IAM Admin
5	Document review completion	IAM Admin

```
// Access review: most and least active users in the window
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-180d
| filter event.kind == "AUDIT_EVENT"
| summarize {events = count(), last_seen = takeMax(timestamp)}, by:{user.id}
| sort last_seen asc
| limit 50
```

```
// Access review: High-activity users (potential over-privileged)
fetch logs, from: now() - 30d
| filter matchesPhrase(log.source, "audit")
| summarize action_count = count(), by: { content }
| sort action_count desc
| limit 50
```

Access Review Checklist

- ☐ All users verified as current employees
- ☐ Access levels appropriate for job function
- ☐ No separation of duties violations
- ☐ Privileged access justified and approved
- ☐ Inactive accounts identified for removal
- ☐ Service accounts still required
- ☐ Review documented and signed off

Review Frequency by Access Level

Access Level	Review Frequency	Approver
Viewer	Annually	Manager
Editor	Semi-annually	Manager + Security
Admin	Quarterly	Director + Security
Account Admin	Monthly	CISO

7. Building Compliance Dashboards

Create dashboards for ongoing compliance monitoring.

Dashboard Components

Section	Metrics	Chart Type
Authentication	Logins/failures over time	Line chart
Access Denied	Failed access attempts	Bar chart

Section	Metrics	Chart Type
User Changes	Create/delete/modify	Table
Admin Activity	Privileged actions	Table
Token Operations	Creates/revocations	Table

```
// Dashboard: Authentication trend (for line chart)
fetch logs, from: now() - 30d
| filter matchesPhrase(log.source, "audit")
| filter matchesPhrase(content, "login")
| summarize logins = count(), by: { day = bin(timestamp, 1d) }
| sort day asc
```

```
// Dashboard: Failed access by day (for bar chart)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-30d
| filter event.kind == "AUDIT_EVENT"
| filter not startsWith(event.outcome, "2") and event.outcome != "success"
| makeTimeseries failures = count(), interval:1d
```

```
// Dashboard: Recent administrative actions (for table)
// Data object corrected 08/12/2026. The Dynatrace audit trail is NOT in
`logs`: this cell used
// `fetch logs | filter matchesPhrase(log.source, "audit")`, and no log.source
on a Grail tenant
// contains "audit" – the filter matched nothing, silently, forever. Platform
audit records live in
// `dt.system.events` with `event.kind == "AUDIT_EVENT"` (265,000+ records
over 7 days here), and
// they are STRUCTURED, so the old `matchesPhrase(content, ...)` string-
scraping is replaced by real
// field predicates. Key fields: user.id, user.organization, event.type
(GET/POST/PUT/PATCH/DELETE/
// CREATE/LOGIN/app.opened), event.outcome (HTTP status or "success"),
authentication.type
// (OAUTH2/TOKEN/NONE), authentication.grant.type, resource (the API path),
origin.address,
// origin.type, request.source, dt.app.id, event.provider.
// Enumerate your own with:
//   fetch dt.system.events, from:-24h | filter event.kind == "AUDIT_EVENT" |
limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "AUDIT_EVENT"
| filter in(event.type, {"POST", "PUT", "PATCH", "DELETE", "CREATE",
"UPDATE"})
| fields timestamp, user.id, event.type, resource, event.outcome
| sort timestamp desc
| limit 25
```

Alert Thresholds

Configure alerts based on audit log patterns:

Condition	Threshold	Severity
Failed logins	> 5 in 10 min same user	Warning
Failed logins	> 20 in 10 min any	Critical
Admin action	Any outside business hours	Warning
Policy change	Any	Info
User deletion	Any	Info
Bulk data access	> 1000 queries/hour	Warning

Next Steps

With compliance monitoring in place, proceed to multi-environment management:

Recommended Path

1. **IAM-08: Multi-Environment IAM** - Scale IAM across environments
2. **IAM-09: Troubleshooting Access Issues** - Debug permission problems

Compliance Checklist

Before moving on, ensure you have:

- ☐ Identified applicable compliance frameworks
 - ☐ Created queries for required audit reports
 - ☐ Established access review schedule
 - ☐ Built compliance monitoring dashboard
 - ☐ Configured alerts for critical events
 - ☐ Documented audit log retention policy
-

Summary

In this notebook, you learned:

- Audit log fundamentals and retention requirements
 - Authentication event monitoring patterns
 - Authorization and access event tracking
 - Configuration change monitoring
 - Compliance-specific query patterns (SOC2, SOX, HIPAA, PCI-DSS)
 - Access review workflows
 - Compliance dashboard components
-

References

- [Audit Logs](#)
 - [DQL Reference](#)
 - [SOC2 Trust Principles](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.